esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

Conception Orientée Objet & Programmation JAVA

Chapitre 7: Les Exceptions

Objectifs du chapitre

- Identifier le rôle des exceptions en Java
- Distinguer if-else et gestion des exceptions
- Utiliser le bloc try-catch pour gérer les erreurs
- Expliquer le cheminement d’une exception
- Différencier throw et throws
- Appliquer le multi-catch pour plusieurs exceptions

Exception

Exception (Définition)

- Une exception en Java est un événement anormal survenant pendant l’exécution d’un programme et perturbant son flux normal.

- Elle signale une erreur ou une situation inattendue (division par zéro, fichier inexistant, valeur nulle) pouvant empêcher le programme de s’exécuter correctement.

- Le mécanisme de gestion des exceptions permet de détecter, intercepter et traiter ces erreurs, assurant ainsi une exécution plus sûre et contrôlée.

Exception
Pourquoi gérer les exceptions ?

Sans gestion des exceptions, le programme peut s’arrêter brutalement.

Une gestion appropriée permet de :

- Éviter les crashs et interruptions inattendues
- Afficher des messages clairs pour informer l’utilisateur
- Rendre l’application plus robuste et fiable

If-Else vs Exceptions quelle différence ?

if-else

- Contrôle le flux normal d’un programme.
- Vérifie des conditions prévisibles et maîtrisées.
- Moins coûteux en termes de performances.

try-catch

- Gère les erreurs imprévues pendant l’exécution.
- Intervient lorsqu’une exception est levée (ex. : fichier manquant)
- Offre une flexibilité pour capturer et traiter les problèmes inattendus.

If-Else vs Exceptions
Exemple comparatif

Avec if-else

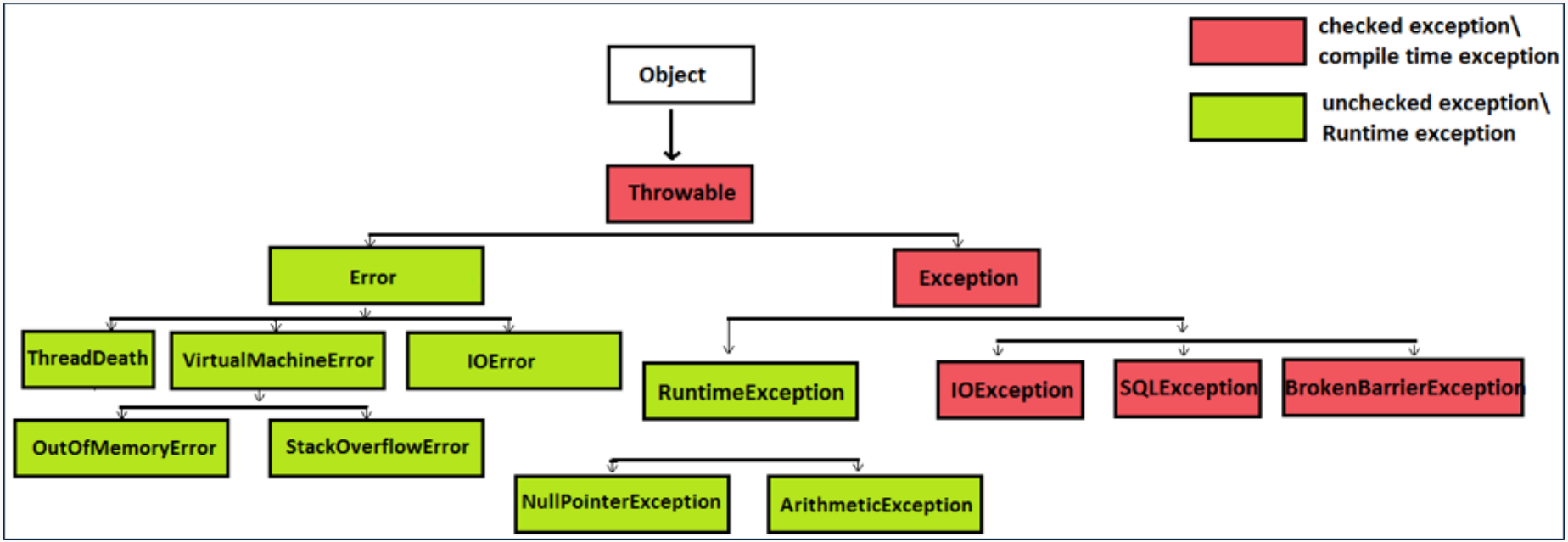
```java
if (b != 0) {
 System.out.println(a / b);
 }
 else {
  System.out.println(
      "Division par zéro interdite");
   }
```

Avec try-catch

```java
try {
    System.out.println(a / b);
} catch (ArithmeticException e) {
    System.out.println(
        "Division par zéro capturée");
}
```

- **if-else** permet d'anticiper, **try-catch** permet de réagir aux situations imprévues.

L’hiérarchie des exceptions



L’hiérarchie des exceptions

L’arborescence des exceptions en Java débute avec la classe Throwable, qui constitue la classe de base de toutes les exceptions

- Les Error indiquent des erreurs critiques de l’environnement d’exécution, généralement non récupérables.
- Les Exception représentent des erreurs moins graves, que le programme peut détecter et gérer.
- RuntimeException est la super-classe des exceptions non vérifiées, regroupant les erreurs courantes à l’exécution.

Erreur vs Exception

Erreur (Error)

- Indique un problème critique que l’application ne peut pas corriger.
- Se produit au niveau de la JVM (ex. : OutOfMemoryError, StackOverflowError).
- Ne doit pas être interceptée ni gérée par un bloc try-catch, car elle échappe au contrôle du programme.

Exception (Exception)

- Indique un problème logique ou opérationnel que le programme peut généralement gérer.
- Se produit pendant l’exécution (ex. : IOException, NullPointerException).
- Doit être interceptée et traitée pour éviter l’arrêt brutal du programme et assurer une exécution stable.

Exception
Quand?

Une **exception** peut être déclenchée de deux manières :

1. **Par la JVM :** lorsqu’une anomalie imprévue survient pendant l’exécution du programme, la machine virtuelle Java génère automatiquement une exception.
2. **Par le programme lui-même :** lorsqu’une condition anormale est détectée, le code peut déclencher explicitement une exception pour avertir la méthode appelante qu’une erreur s’est produite.

Exception exemple

Bien que ce code **ne contienne aucune erreur de compilation**, une **exception sera levée à l'exécution**, car la **division par zéro est interdite** en Java et provoque une **ArithmeticException**.

```
public class TestException{
    public static void main(String[] args) {
        int x = 5;
        int y = 0;
        System.out.println("Avant incident");
        System.out.println("La division de x/y
            est :" + x / y);
        System.out.println("Après incident");
    }
}
```

Exception
exemple

Dès l'exécution de l'instruction `x / y`, un incident s'est produit :

- la JVM a lancé une exception de type `ArithmeticException`, c'est-à-dire qu'un objet de cette classe a été créé pour signaler l'erreur.
- Cette `exception` interrompt immédiatement l'exécution normale du programme et empêche le reste du code de s'exécuter.

```
Output:
Avant incident
Exception in thread "main" java.lang.ArithmeticException:
/ by zero at TestException.main(TestException.java:7)
```

erreur de compilation & erreur d'exécution

Une erreur de compilation est une erreur qui **empêche le programme** d’être **compilé**, car elle viole **les règles du langage Java** (ex. : faute de syntaxe, type incompatible).

En revanche, **une erreur d’exécution** se produit au **moment de l’exécution (runtime)** et se manifeste par la **levée d’une exception** générée par la JVM.

Les exceptions vérifiées et non vérifiées

Les exceptions en Java sont réparties en deux grandes catégories :

- **Exceptions vérifiées (Checked Exceptions)**
- **Exceptions non vérifiées (Unchecked Exceptions)**

Ces deux types jouent un rôle **fondamental** dans la détection, la gestion et la **propagation des erreurs**, influençant directement la fiabilité et la robustesse des programmes Java.

Les exceptions vérifiées

- **Les exceptions vérifiées** (checked exceptions) correspondent généralement à des **événements prévisibles** qu'une application doit être capable **d'anticiper et de gérer**.
- Elles doivent être **traitées explicitement** dans le code, soit en les capturant avec un **bloc** `try-catch`, soit en les déclarant dans la signature de la méthode à l'aide du **mot-clé** `throws`.

Les exceptions vérifiées exemple

Le programme tente d'ouvrir le **fichier "file.txt"** pour lecture. Si le fichier n'existe pas ou si des problèmes d'autorisation surviennent, une **FileNotFoundException** est levée.

Cette **exception** étant **vérifiée**, elle doit impérativement être **gérée (via un bloc try-catch ou déclarée avec throws)** afin de permettre la compilation du programme.

Les exceptions non vérifiées

- **Les exceptions non vérifiées** (unchecked exceptions) correspondent généralement à des **événements imprévus** que l'application ne peut pas anticiper.
- Elles sont souvent évitablement liées à des erreurs de programmation (ex. : **NullPointerException, ArithmeticException**) et ne sont pas soumises à une vérification à la compilation, laissant au développeur la responsabilité de les prévenir ou de les gérer.

Les exceptions non vérifiées exemple

Dans cet exemple, le programme tente d'accéder à un élément inexistant du **tableau numbers à l'indice 4**, ce qui déclenche une `ArrayIndexOutOfBoundsException`.

Ce code compile sans **erreur**, mais provoque une **exception à l'exécution**, interrompant le flux normal du programme.

```java
public class TestException{
   public static void main(String[] args) {
    int[] numbers = { 1, 2, 3 };
    System.out.println(numbers[3]);
   }
}
```

```
Output:
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException:
Index 3 out of bounds for length 3 at
TestException.main(TestException.java:5)
```

Gestion des Exceptions

1. **La gestion des exceptions** repose sur l'anticipation des comportements risqués d'une méthode.
2. Si **une méthode peut provoquer une anomalie**, il est nécessaire d'écrire **explicitement** du code pour en tenir compte.
3. Lorsqu'une méthode peut présenter un tel comportement, on dit qu'elle peut générer **(lancer ou lever)** une **exception**, permettant ainsi au programme de **détecter et gérer l'erreur**.

Gestion des Exceptions

Au cœur de la gestion des exceptions en Java se trouvent cinq concepts clés : `try`, `catch`, `finally`, `throws` et `throw`

La gestion des exceptions peut s'effectuer de deux manières :

- **Gestion active :** le programme prend l'initiative de gérer les exceptions de manière anticipée.
- **Gestion passive :** le programme laisse l'exception se propager, la méthode appelante devant alors la gérer.

Gestion active (try-catch)

Le **bloc** **try-catch** permet de **capturer** et **gérer** une **exception** de manière appropriée.

Le code susceptible de provoquer une exception est placé dans le bloc **try**.

Si une exception survient, l’exécution **du bloc** **try** est interrompue, et le contrôle est transféré au **bloc** **catch**, où le type d’exception est vérifié et une action corrective peut être effectuée.

Gestion active (try-catch) exemple

La gestion de l’exception n'interrompt pas l’exécution du reste de programme.

```
public class TestException{
   public static void main(String[] args) {
     try{
        System.out.println("Avant incident");
        int[] numbers = { 1, 2, 3 };
        System.out.println(numbers[3]);
    }catch(ArrayIndexOutOfBoundsException ex){
        System.out.println("incident");
}
    System.out.println("Après incident");
   }
}
```

Output:

```
Avant incident
incident
Après incident
```

Gestion active (try-catch) exemple

On peut déclarer plusieurs **blocs catch**, mais ils doivent être organisés du cas le plus **spécifique** au cas le plus **général**.

```
public class TestException{
   public static void main(String[] args) {
    try{
        int[] numbers = { 1, 2, 3 };
        System.out.println(numbers[3]);
    }catch(Exception ex){
        System.err.println(ex.getMessage());
    }catch(ArrayIndexOutOfBoundsException e){ //jamais appelé
        System.err.println(e.getMessage());
    }
     }
}
```

Gestion active (finally)

Le bloc **finally** est utilisé pour contenir du code qui sera exécuté quel que soit le résultat de la tentative **(try)** et de la gestion d'exception (catch).

Seul l'appel de **System.exit()** empêchera l'exécution du bloc **finally**.

```
public class TestException{
   public static void main(String[] args) {
    try{
        int[] numbers = { 1, 2, 3 };
        System.out.println(numbers[3]);
    }catch(ArrayIndexOutOfBoundsException ex){
        System.err.println(ex.getMessage());
    }finally{
        System.out.println("Le bloc finally s'exécute toujours");
}
   }
}
```

Gestion active (finally)

- La clause **finally** peut être utilisée avec ou sans des clauses **catch**.
- Il est couramment utilisé pour effectuer des opérations de nettoyage, telles que la fermeture de fichiers ou la libération de ressources.

```
public class TestException{
   public static void main(String[] args) {
    File file = null;
    try{
        file = new File("temp.txt");
        System.out.println(file.getName());
    }finally{
        if (file != null) {
        file.delete();
    }
}
   }
}
```

Gestion passive (throws)

- Le **mot-clé** `throws` est utilisé dans la déclaration d'une méthode pour indiquer que **la méthode peut lancer certaines exceptions.**

- Les méthodes qui appellent cette méthode **doivent** soit capturer **ces exceptions avec un bloc** `try-catch` ou déclarer à leur tour qu'elles **peuvent lancer ces exceptions avec le mot-clé** `throws`**.**

Gestion passive (throws) exemple

```
public class TestException {
    public int divide(int x, int y) throws ArithmeticException {
        return x/y;
    }
    public static void main(String[] args) {
        Util u = new Util();
        try{
            int x = u.divide(10,0);
        }catch(ArithmeticException ex){
            System.err.println(ex.getMessage());
    }
}
}
```

Lancer une exception (throw)

Vous pouvez lancer manuellement une exception en utilisant le mot-clé **throw** suivi de l'instance de l'exception que vous souhaitez lever.

```
public class TestException {
    public static void main(String[] args) {
        try{
            throw new Exception("Une erreur s'est produite !");
        }catch(ArithmeticException ex){
            System.out.println("Erreur: " + e.getMessage());
        }
    }
public void tester(int x, int y) throws OhNoException{
    throw new OhNoException("Oh no !");
    }
}
```

Les exceptions personnalisées

- il est possible de créer des classes d'exceptions personnalisées en étendant les classes de base **Throwable**, **Error** ou **Exception**.
- Ces classes d'exceptions personnalisées peuvent être utiles pour **signaler des erreurs spécifiques** à une application ou à une bibliothèque particulière.

```
public class OhNoException extends Exception{
    public OhNoException(){
        super();
    }
    public OhNoException(String message){
        super(message);
    }
}
```

Exception et bonnes pratiques

Il est important de ne pas utiliser les exceptions pour contrôler le flux normal d'un programme.

Pour une gestion efficace des **erreurs**, il est recommandé de :

- Ne pas masquer **les erreurs silencieusement avec un bloc catch(Exception e) {} vide**.

- Toujours consigner ou afficher un message utile afin de faciliter le diagnostic et le débogage.

Merci pour votre attention